01. ORIGINS, TYPES, MEMORY

курс лекций по информатике и программированию
для студентов первого курса ИТИС КФУ (java-поток)
2023/2024

М.М. Абрамский

кандидат технических наук, доцент кафедры программной инженерии

Формальности (еще раз)

- 18 лекций на 2 семестра
- в феврале и июне/июле – письменный экзамен (теория + практика)

- все презентации все выкладываю по ссылке: https://disk.yandex.ru/d/GI6U_ybUggi8oQ
- лекции (видео) в онлайн не дублирую*
- остается возможность задавать мне вопросы, чтобы я их разобрал на лекции либо мне на почту mabramsk@kpfu.ru, либо через старосту. До понедельника 23:00 перед средой следующей лекции

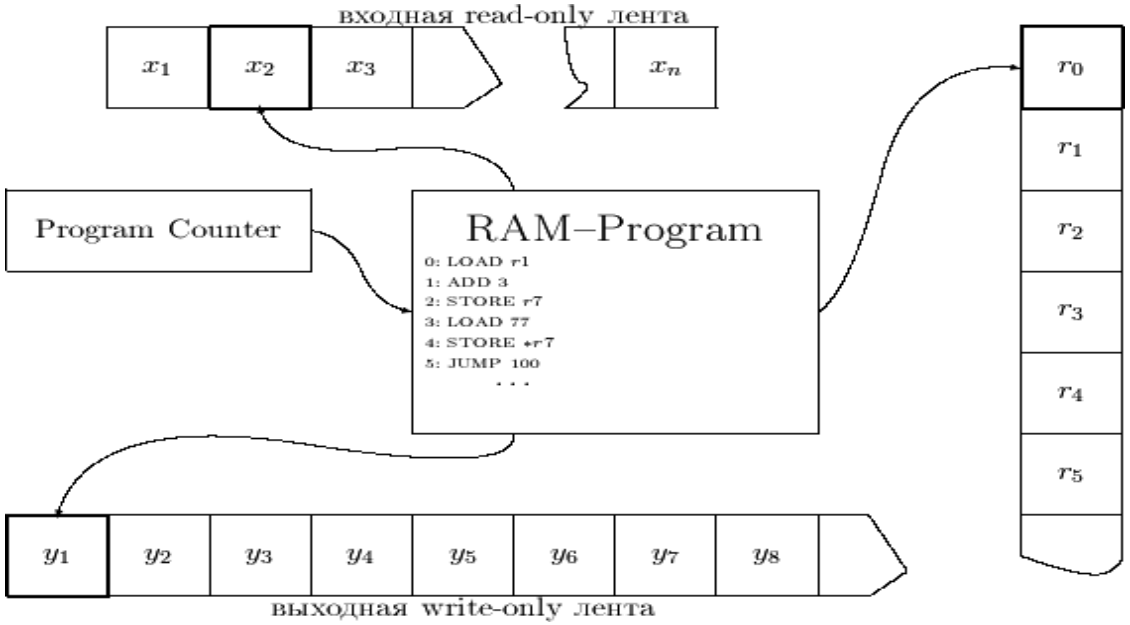
Мы остановились тут (см. презентацию консультации 1)

1. Кризис математики привел к формализации понятия «алгоритм».

2. Появились технологии, позволяющие строить вычислительные устройства.

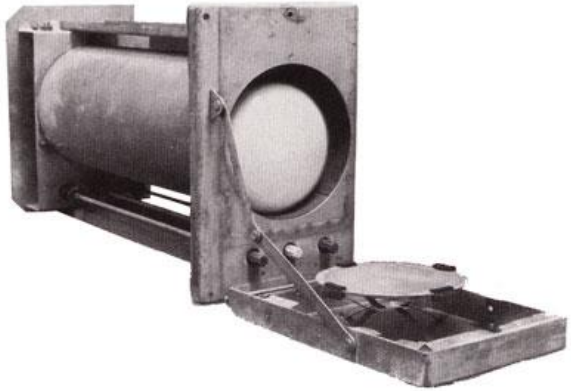
3. Тезис Чёрча-Тьюринга и теорема об универсальной машине обусловили появление **программирования.**

Random Access Machine (1960е)



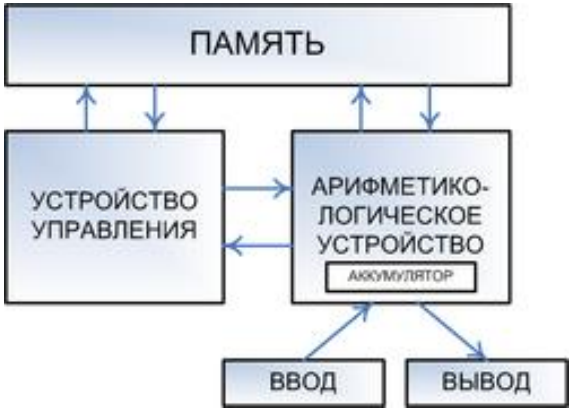
Кстати, другая расшифровка RAM

Random Access Memory – Оперативная память (память с произвольным доступом)

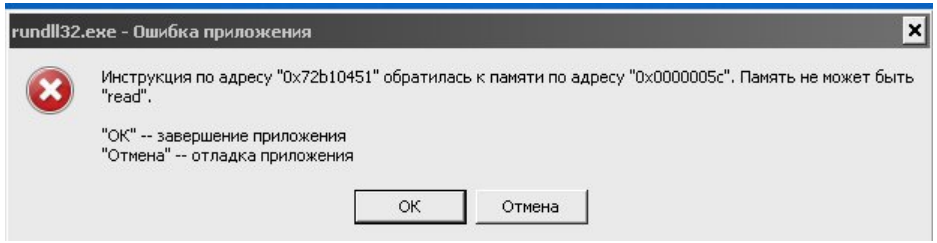


Трубка Уильямса (1946-1948) – первая память с **произвольным** доступом

Джон фон Нейман (1903 - 1957)



1101001010
1010010101
0010101010
10101001111
0110



Принципы архитектуры фон Неймана

- Однородность памяти (команды и данные хранятся в общей памяти)
- Адреса (память – пронумерованные ячейки)
- Программное управление (программа – последовательность команд)
- Двоичное кодирование

«Физика»

- Компьютер – физическое устройство

- Если мы всю информацию кодируем в числах, то должен быть какой-нибудь физический процесс, умеющий кодировать в себе 10 разных состояний

  (это много)

Двоичная система счисления

Двоичная – т.е. разрешены только цифры 0 и 1 (меньше 2)

Все числа – наборы 0 и 1

$1_2 = 1_{10}$

$10_2 = 2_{10}$

$11_2 = 3_{10}$

$100_2 = 4_{10}$

$101_2 = 5_{10}$

$110_2 = 6_{10}$

$111_2 = 7_{10}$

Что вообще такое число?

$$1234 =$$

$$1000 + 200 + 30 + 4 =$$

$$1 \cdot 1000 + 2 \cdot 100 + 3 \cdot 10 + 4 \cdot 1 =$$

$$1 \cdot 10^3 + 2 \cdot 10^2 + 3 \cdot 10^1 + 4 \cdot 10^0$$

Основание системы счисления

Любое натуральное число можно разбить на сумму степеней 2

$$45_{10} = 32 + 13 = 32 + 8 + 5 =$$

$$32 + 8 + 4 + 1 = 2^5 + 2^3 + 2^2 + 2^0 =$$

$$101101_2$$

5 4 3 2 1 0

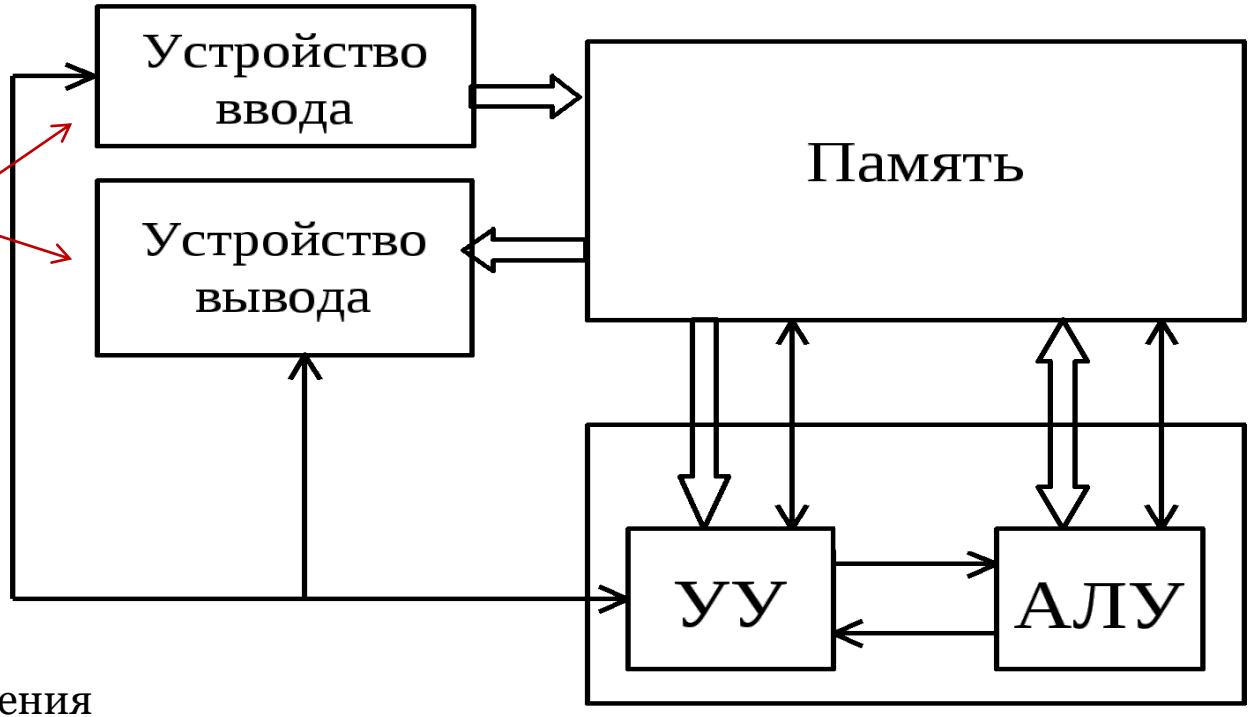
Почему программисты любят числа, равные степеням двойки

- $2^k = 100 \ldots 0$

- $4 = 2^2 = 100_2$
- $8 = 2^3 = 1000_2$
- $16 = 2^4 = 10000_2$

Архитектура фон Неймана

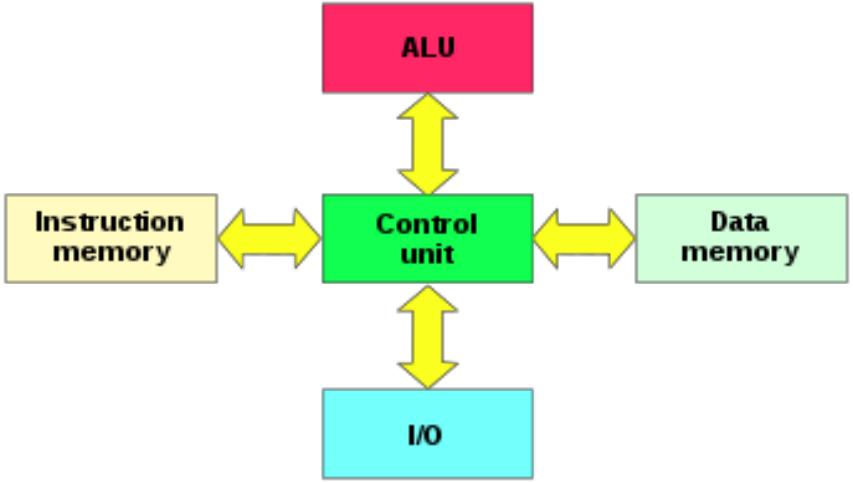
Не только ввод/вывод, а любые внешние устройства, работающие с данными (платы, жесткий диск, порты, устройства ввода/вывода и т.д.)



Процессор

УУ – устройство управления
АЛУ – арифметико-логическое устройство
Память – оперативная память

Гарвардская архитектура



256

- Комбинаторика: 2^8 = 256
- Или в лоб:

```
00000000 = 0
00000001 = 1
00000010 = 2
…
11111110 = 254
11111111 = 255
```

Но как быть с отрицательными числами?

Число со знаком

- Первый бит – знак (0 – плюс, 1 – минус).
- Получается на само число остается на 1 бит меньше.
- byte – целые числа из [-128; 127]
  - 128 = 2^7, хотя в байте 8 бит
  - По аналогии с int, short, long

boolean

true, false

значения выражений сравнения имеют тип boolean.

Логические операции:

- !x – «НЕ x»,
- x && y, x & y – «x И y»,
- x || y, x | y – «x ИЛИ y»

Разница:

&, | - считают всё (оба аргумента).

&&, || - вычисление останавливается, если результат уже понятен.

- Ленивые (lazy) выражения.

Поэтому

НИКОГДА не делайте сравнение 2х вещественных значений с помощью:

- ==
- !=
- <=
- >=

! Подумайте, как сравнивать вещественные числа друг с другом правильно

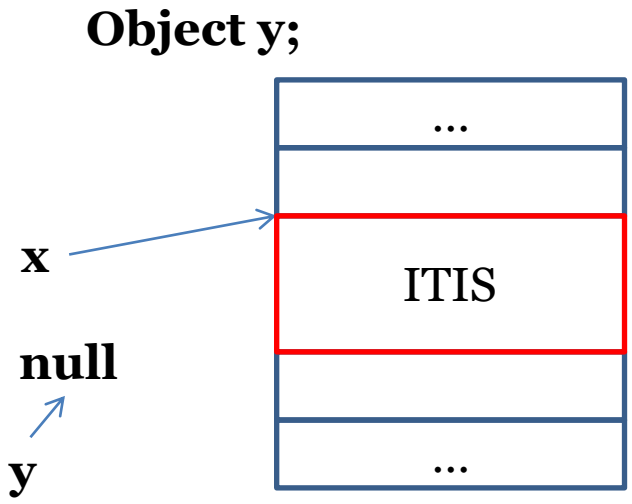
«Еще сахару»

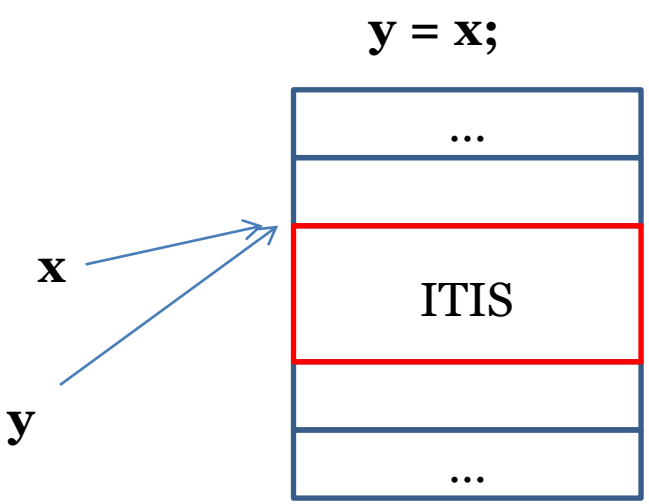
+=, -=, *=, /= и т.д.

x += y

Для Java: Это не “x = x + y”, а “x = (тип x)(x + y);”

Память и ссылочные переменные





При присваивании не происходит копирования данных. Копируется только ссылка. (*логично – у вас у всех есть ссылка на универ, в котором вы учитесь, но универ при этом не дублируется*)

Хранение значений переменных в ОП

- **Стек (Stack)** – место хранения переменных примитивного типа
  - Небольшой размер (Stack Overflow)
  - Быстрые операции добавления и удаления

- **Куча (Heap)** – место хранения данных ссылочного типа
  - Динамически выделяемая память (большого размера)

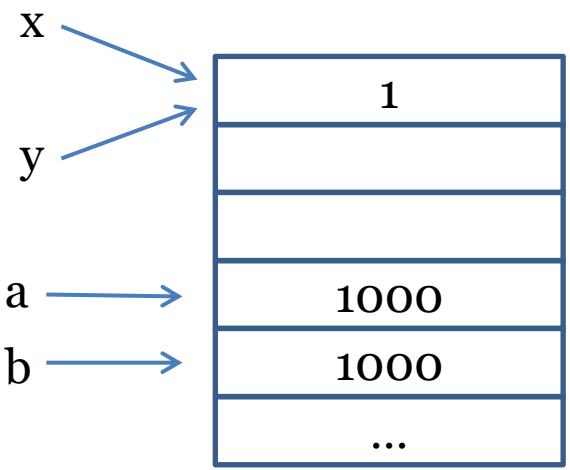
Помните с консультаций?

```
Integer x1 = 123;
Integer x2 = 123;
x1 == x2; // true

Integer x1 = 1234;
Integer x2 = 1234;
x1 == x2; // false
```

Кэш

«Если не создан – создай»

«А если создан – не создавай, а возврати имеющийся»



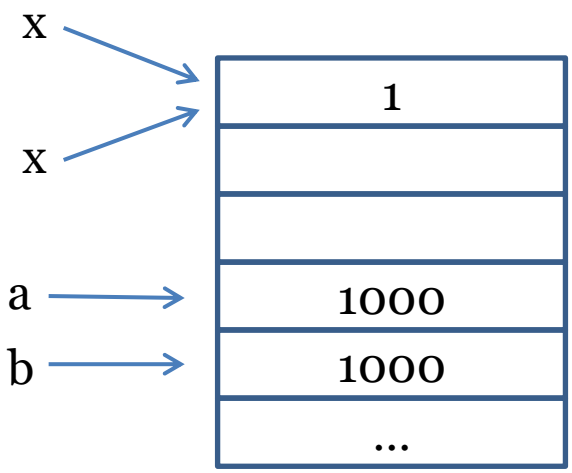

```
Integer x = 1;
Integer y = 1;

Integer a = 1000;
Integer b = 1000;
```

Сравнение

== означает
«является ли одним и тем же»

x и y – одно и то же

a и b – нет

Поэтому для сравнения ссылочных нужно использовать

a.equals(b)